SOAR OPERATIONS PLATFORM

Project Documentation and Security Engineering Review

Repository snapshot reviewed: 27 August 2026
Scope: application code, configuration, web UI, tests, deployment and operating guidance

Assessment posture
Simulation-first laboratory platform. Suitable as a controlled prototype; not a production reference architecture without the remediation and hardening described in this report.

Executive summary

SOAR is a single-process FastAPI application that receives authenticated detections, enriches source IPs through AbuseIPDB, calculates an explainable risk score, gates high-impact actions through operator approval, simulates or applies device rules, and records state transitions in a SHA-256 hash chain.

Overall assessment: the code demonstrates deliberate safety engineering for a lab system: bounded inputs, role checks, signed sessions, conservative enrichment failure behavior, atomic approval claims, response allow-listing, dual simulation flags, durable response intent, and audit-chain verification. The strongest design theme is fail-closed network response.

Residual risk: production use would carry material identity, concurrency, availability, database, CSRF, audit durability, and network-change risks. The shipped example administrator password remains active unless overridden. API documentation and health data are public. Login throttling, sessions, enrichment caching, and audit serialization are process-local. The responder creates persistent ACL/filter changes without a first-class rollback workflow.

Area	Assessment	Key reason
Response safety	Strong for lab	Approval state, IPv4 allow-list, supported devices, simulation defaults
Application security	Moderate	Good headers and RBAC; default credential and no explicit CSRF token
Data integrity	Moderate	Hash chain and anchors; writable local storage is not immutable
Reliability	Moderate-low	Synchronous external I/O, SQLite, process-local state
Production readiness	Low	Single-process assumptions, no migrations, MFA/IdP, queue, HA, or rollback engine

Priority actions

- P0: refuse startup outside an explicit development mode when default credentials or an ephemeral/example session secret are present.
- P0: keep live response disabled until device-specific rollback, change authorization, command validation, and end-to-end lab tests exist.
- P1: add explicit CSRF tokens for browser state changes and protect or disable interactive API documentation.
- P1: move production state to PostgreSQL, serialize audit writes transactionally, introduce migrations, and use a durable job queue.
- P1: adopt named federated identity with MFA and centralized, shared rate limiting.

1. System purpose and scope

The project is a compact security orchestration, automation, and response control plane for authorized cyber-range and laboratory environments. It combines intake, intelligence, decision support, governance, enforcement, and evidence in a single deployable service.

Primary capabilities

- Authenticated IPv4/IPv6 detection intake with severity and evidence validation.
- AbuseIPDB reputation lookup with five-second timeout, structured failure records, and in-memory TTL cache.
- Deterministic weighted scoring and human-readable reasoning.
- Human approval/rejection with an atomic event-state claim.
- Cisco IOS and Juniper Junos block-command generation; live access through Netmiko.
- Server-rendered console plus five-second JSON polling with ETag support.
- Tamper-evident audit log, local chain-head anchor, and optional external HMAC-protected anchor.

Explicit boundaries

- The responder is IPv4-only even though detection intake accepts IPv6.
- Default storage is SQLite; cache and throttling are held in one process.
- Enrichment and response execute synchronously in request handling.
- There is no migration framework, message broker, HA design, federated identity, or device rollback subsystem.
- The repository documentation correctly describes the service as simulation-first and not directly internet-safe.

2. Architecture and component map

```
Browser / API client
↓ signed cookie or HTTP Basic
FastAPI (main.py)
↓ event → enrich → decide → approve → respond
SQLAlchemy / SQLite    AbuseIPDB    Cisco / Juniper device
↓
Hash-chained audit log + optional external anchor
```

File / area	Responsibility
main.py	Application lifecycle, auth, RBAC, validation, endpoints, orchestration, state transitions, retention.
models.py	SQLAlchemy schema for events, enrichment, decisions, approvals, response jobs, audit log, anchor.
enrichment.py	AbuseIPDB client, cache, error normalization and persistence.
decision_engine.py	Configuration validation, scoring, confidence, action mapping, approval rules.
responder.py	Allow-list enforcement, command generation, simulation, Netmiko execution.
audit_log.py	Canonical hashing, audit creation, local/external anchors, verification helpers.
templates/ + static/	Login and operations UI; escaping, polling, search, dialogs, role-aware actions.
tests/	API regressions, validation/auth, approval atomicity, response state, responder and audit security.

Runtime characteristics

- FastAPI synchronous route handlers use request-scoped SQLAlchemy sessions.
- Configuration and identity records are loaded at process start.
- A custom middleware adds security headers and request IDs; TrustedHostMiddleware restricts Host headers.
- Static assets are local, so the CSP can avoid external script/style origins.

3. Data model and lifecycle

Entity	Important fields / constraints
Event	Source IP, type, severity, evidence, timestamp, status. Root of the workflow.
EnrichmentResult	Event link, reputation fields, raw response, TTL, error.
Decision	Unique event_id; action, confidence, score, approval flag, reasoning JSON text.
Approval	Unique event_id; decision, status, actor, reason, timestamps.
ResponseJob	Unique event and idempotency key; attempts, claim/result/error.
AuditLog	Actor/action/state/reasoning plus previous and current SHA-256 hash.
AuditAnchor	Singleton local record of latest entry ID/hash.

State machine

```
new -> enriched -> decided -> pending_approval -> responding -> responded
+-> rejected +-> closed / response_failed
unexpected persisted-stage failure -> processing_failed
```

The approval endpoint conditionally updates only an event in `pending_approval`. A one-row update grants ownership; competing or repeated reviews receive 409. Response jobs persist intent before device I/O and reuse completed results through a stable idempotency key.

Data-management observations

- Foreign keys are declared, but SQLite foreign-key enforcement is not explicitly enabled through PRAGMA; deployment should verify enforcement.
- Schema creation uses create_all and has no migration/version management.
- Raw evidence and enrichment responses may be sensitive. Retention can minimize terminal-event evidence while preserving lifecycle metadata.
- The database stores device output in response job JSON until retention minimization applies.

4. API and operator interface

Method / route	Authorization	Behavior
GET /login	Public	Render login page.
POST /login	Public + throttle	Validate credentials; set signed HttpOnly Strict cookie.
POST /logout	Public	Clear cookie; browser SameSite policy limits cross-site use.
GET /	Session	Render role-aware console.
GET /dashboard/data	Viewer+	Recent events, decisions, approvals and audit status; ETag.
POST /detections	Operator+	Run complete synchronous detection pipeline.
POST /approvals/{id}	Operator+	Atomic approval/rejection and possible response.
POST /response-jobs/{id}/retry	Admin	Retry eligible durable response job.
GET /events/{id}	Viewer+	Event, enrichment, reasoning, approval and audit context.
GET /health	Public	Database and recent audit-chain check.

Input controls

- Pydantic validates IP syntax, event type length and nonblank content, severity 1-5, evidence length up to 10,000, and timezone-aware timestamps.
- Rejections require a nonblank reason up to 2,000 characters.
- Browser output is escaped before insertion into innerHTML; Jinja autoescaping protects server-rendered values.
- The dashboard uses same-origin fetches, abortable polling, ETags, visibility-aware scheduling, and authorization-aware review controls.

5. Decision and approval logic

The default composite score is deterministic and configuration-driven:

```
risk = 100 × (abuse reputation × 0.40 + normalized severity × 0.35 + normalized report count × 0.25)
```

- Abuse score is clamped to 0-100.
- Severity is clamped and divided by five.
- Report count saturates at 200.
- Failed enrichment contributes neutral reputation/report inputs, then triggers conservative approval logic.
- Block begins at 75; monitor at 50; otherwise ignore.

Confidence begins at 0.5, drops to 0.4 on enrichment failure, and rises for report history and very fresh results, capped at 0.99. The reasoning record captures normalized inputs, weights, confidence factors, result, and approval reasons.

Governance behavior

- Approval safety rules run before action-specific automatic approval.
- Block actions require approval by default.
- High-severity and uncached/failed enrichment can force review.
- Monitor can auto-approve only below its configured ceiling; ignore is normally automatic.
- Configuration loading validates required sections, exact risk-weight keys, weight ranges/sum, and threshold ordering.

6. Security controls verified in code

Control	Implementation evidence / assessment
Password storage	scrypt with random 16-byte salt; constant-time verifier comparison. Plaintext configuration remains supported and should be prohibited in shared environments.
Session integrity	HMAC-SHA256 signed JSON with expiry; HttpOnly, SameSite=Strict and configurable Secure cookie.
Authorization	Admin/operator/viewer roles enforced through FastAPI dependencies.
Login abuse	Bounded in-memory attempts keyed by client/account; useful for one process, not distributed defense.
Browser hardening	CSP, frame denial, nosniff, no-referrer, permissions policy, COOP, conditional HSTS and no-store pages.
Host validation	TrustedHostMiddleware with environment allow-list.
Network safety	Target and device IPv4 allow-list; supported type check; both dry-run and simulation must be explicit YAML false for live I/O.
Audit integrity	Canonical chain includes event ID and state; sequence, hash, local anchor and optional HMAC external anchor checks.
Failure disclosure	External errors generally become controlled messages; device exception details are not returned.

Positive design detail

The responder verifies that the event is already in the claimed `responding` state before generating or sending commands. It repeats target/device allow-list checks immediately before opening a socket and guarantees a disconnect attempt.

7. Findings and risk register

ID / severity	Finding	Recommendation
F-01 Critical (deployment)	Example admin password is a functional fallback; startup only warns.	In non-development mode, fail startup unless a strong hash and persistent session secret are supplied.
F-02 High	Live device changes lack first-class rollback, reconciliation, expiry, and post-change validation.	Add device-specific rollback plans, command diff/validation, change IDs, TTL rules, verification and tested recovery.
F-03 High	SQLite, process-local rate limits/cache, and audit creation assume one worker.	Use PostgreSQL, transactional advisory locking/serialized audit append, Redis-backed limits/cache, and a durable queue.
F-04 Medium	Browser state changes have no explicit anti-CSRF token. SameSite=Strict is the primary control.	Add synchronizer or signed double-submit CSRF tokens to login/logout/approvals and browser-origin mutations.
F-05 Medium	Public /docs, /redoc, /openapi.json and /health increase reconnaissance surface.	Disable or authenticate docs in shared deployments; minimize public health detail.
F-06 Medium	Audit chain is tamper-evident, not immutable; local DB and anchor may share control plane.	Require separately protected HMAC anchor and append-only remote logging; alert and stop response on failure.
F-07 Medium	Synchronous enrichment/device I/O ties request capacity to external systems.	Queue work, enforce timeouts/retries/backoff, expose job state, and isolate device workers.
F-08 Medium	No migration framework and uncertain SQLite FK enforcement.	Adopt Alembic, enable/check foreign_keys, test upgrades and backups.
F-09 Low	Session tokens are stateless with no per-session revocation or key rotation scheme.	Add session store or short-lived tokens with rotation/versioning and revocation.
F-10 Low	A client-supplied X-Request-ID is accepted if bounded but not character-normalized.	Restrict to safe printable identifier syntax before logging/returning.

8. Threat model summary

Threat actor / failure	Likely path	Existing mitigation	Gap
Credential attacker	Default/weak password, brute force	scrypt, constant-time checks, local throttle	Default fallback; no MFA/shared limits
Malicious operator	Approve harmful block, view evidence	RBAC and audit trail	Operator can authorize; no four-eyes or scoped tenancy
Compromised app host	Rewrite DB/anchor, steal device secret	Optional external HMAC anchor	Host env and local anchor remain exposed
Untrusted detection sender	Oversized/malformed evidence, XSS	Auth, bounds, IP parsing, escaping, CSP	Raw evidence remains sensitive and synchronous load is possible
External-service failure	AbuseIPDB timeout/rate limit	Timeout and recorded neutral failure	Request latency; no circuit breaker/queue
Concurrent reviewer	Double approve/respond	Atomic status claim, response idempotency	Broader multi-process audit serialization remains
Misconfiguration	Disable safety via YAML strings/typos	Only literal false disables safeguard	Allow-list can still be overly broad
Device/network failure	Partial ACL change or bad interface	Supported templates, error handling	No transactional rollback or config verification

9. Deployment and operating model

Recommended deployment tiers

Tier	Permitted posture
Developer laptop	Loopback bind, simulation on, disposable SQLite, test credentials only.
Shared cyber range	TLS reverse proxy, named users, secure cookies, restricted ingress, protected DB/anchor, one worker, simulations reviewed.
Live operational network	Not approved on current architecture. Requires the production controls and response-governance program below.

Pre-live control gates

- Named accountable users through an IdP with MFA; no shared admin account.
- Secrets manager for AbuseIPDB, session keys and device credentials; defined rotation.
- Device command review for exact OS/version/interface and a proven rollback command.
- Narrow target and device allow-lists; deny public/routable ranges unless explicitly authorized.
- Separated response worker identity with minimum device privileges.
- Backups and restore drill; independent external audit anchor and centralized logs.
- Change ticket correlation, two-person approval for blocking, maintenance windows, and incident owner.
- Load, concurrency, failure-injection and penetration testing in an isolated replica.

Monitoring

- Availability and latency of health, detection and approval paths.
- Counts and age of pending_approval, processing_failed, response_failed and failed response jobs.
- Login failures, throttle triggers, unusual approval actors and source networks.
- AbuseIPDB errors and cache effectiveness.
- Audit verification failure, anchor mismatch, database growth/capacity, backup age.
- Device connection, command and commit results - while redacting secrets and sensitive output.

10. Verification and test assessment

The repository contains focused tests for API regressions, authentication and validation, approval atomicity, block approval, responder security, response state, audit integrity, dashboard caching, and pipeline failure recovery.

Check	Observed result
Source inventory	6,836 lines across application, UI, documentation, tests and dependency lockfile were inventoried; generated/vendor directories were excluded from semantic review.
Python compilation	Core modules compiled successfully.
Dependency consistency	The active virtual environment reported no broken installed requirements.
Test suite - first invocation	Collection failed because the project root was not on the import path in that invocation.
Test suite - adjusted import path	Execution did not complete within the review window and was interrupted after no progress output. This report does not claim a passing suite.
Static inspection	Security-sensitive sinks, secrets, dynamic HTML, subprocess/eval patterns and network execution paths were reviewed. No eval, shell execution, unsafe YAML loader, TLS verify disable, or pickle use was identified.

Interpretation

The test design appears materially security-aware, but a clean, reproducible CI pass is a release gate. The local collection behavior should be fixed through packaging or pytest pythonpath configuration, and hanging/slow tests should be isolated with per-test timeouts and verbose CI diagnostics.

Suggested CI gates

- pytest with deterministic environment and timeouts; coverage for critical state transitions.
- Ruff/formatter and JavaScript syntax checks.
- Dependency audit against the locked hashes and published SBOM.
- Secret scanning, SAST, container/image scan, and license policy.
- Migration test from last release plus SQLite/PostgreSQL integration tests as applicable.

11. File-by-file documentation

Path	Security-relevant notes
README.md	Accurate overview, quick start, routes, states, safety and configuration. Clearly warns about lab scope.
DEPLOYMENT.md	Single-instance deployment, TLS, simulation, persistence, validation and rollback guidance.
docs/ARCHITECTURE.md	Component, request, data, approval, audit and scaling design.
docs/SECURITY.md	Trust boundaries, implemented controls, limitations and hardening.
docs/OPERATIONS.md	Daily workflow, monitoring, failure triage, maintenance.
config.yaml	Weights, thresholds, approval rules and dual responder safeguards; strict loader validation.
.env.example	Operational variables, but should include explicit session secret, cookie-secure, device interface and external anchor path examples.
dashboard.html	Tiny redirect/compatibility artifact; primary UI is templates/dashboard.html.
requirements*.txt / lock	Runtime/development separation and hashed lock intent. Maintain automated vulnerability review.
scripts/hash_password.py	Interactive scrypt verifier generator; imports main, which may trigger application initialization side effects. Consider isolating crypto helpers.
.github/	CI/release automation is present in the working tree but untracked; review and commit intentionally before relying on it.
SOAR	Tracked repository artifact requiring owner clarification; verify purpose and exclude binary/generated content if accidental.
scripts/10	Untracked artifact requiring owner clarification; do not ship without review.

12. Remediation roadmap

Horizon	Deliverables
Immediate - before shared use	Fail on insecure defaults; set persistent session secret; TLS + Secure cookie; restrict hosts/ingress; disable docs; narrow allow-list; confirm simulation; run tests reproducibly.
30 days	Explicit CSRF; IdP/MFA plan; Alembic; shared rate limiting; external HMAC anchor; centralized logging; secrets manager; backup/restore exercise.
60-90 days	PostgreSQL; durable async jobs; serialized audit append; response worker isolation; command validation, rollback and expiry; two-person approval for live blocks.
Before production	Threat-model sign-off, penetration test, capacity/failure tests, disaster recovery, monitoring/SLOs, runbooks, ownership and formal authorization.

Acceptance criteria

- No service start with known/default credentials in shared or production mode.
- All browser mutations reject missing/invalid CSRF tokens.
- Exactly-once or safely idempotent response behavior demonstrated across restart and concurrent workers.
- Audit tampering/truncation triggers alerting and blocks further automated response.
- Device change is verified and reversible, with preserved change/approval identity.
- Clean CI run from a fresh checkout using the locked dependency set.

Appendix A - Configuration reference

Setting	Purpose / security note
DATABASE_URL	SQLite default; use controlled absolute location. Production should use a managed transactional database.
ABUSEIPDB_API_KEY	External intelligence credential; store in secret manager.
ABUSEIPDB_CACHE_TTL_MINUTES	Positive process-local cache duration.
SOAR_ADMIN_USERNAME / PASSWORD / HASH	Hash preferred. Default plaintext fallback is unacceptable outside development.
SOAR_USERS_JSON	Local accounts and roles; hashed records preferred.
SOAR_SESSION_SECRET / TTL / COOKIE_SECURE	HMAC key, lifetime and HTTPS-only cookie. Secret must be stable and random.
SOAR_ALLOWED_HOSTS	Host-header allow-list.
SOAR_LOGIN_ATTEMPT_LIMIT / WINDOW	In-memory throttle; add shared/proxy layer.
SOAR_EVIDENCE_RETENTION_DAYS	Minimizes evidence for old terminal events.
LAB_ALLOWED_IPS	Critical safety boundary; keep narrowly scoped.
LAB_DEVICE_*	Device endpoint, type, interface and credentials.
AUDIT_EXTERNAL_ANCHOR_PATH / HMAC_KEY	Independent chain-head evidence; protect separately.
SOAR_RESPONSE_MAX_ATTEMPTS	Bounds durable response retries.
responder.dry_run / simulation_mode	Either true prevents live connection; only literal YAML false disables.

Appendix B - Review notes

This is a source-assisted security engineering review, not a penetration test or formal certification. Findings are based on the repository state visible on 27 August 2026, including uncommitted changes. Existing source files were not modified. The generated PDF is the only intended deliverable.

Repository status contained modified and untracked files at review time. Any release assessment should be repeated against an immutable commit hash after maintainers decide which working-tree changes belong in the release.

End of report